

분야 AI 활용 학습 구분 AI 활용 학습시간 40시간

Python 응용: API 활용 국내 여행지 추천 프로그램 개발

문제기술

기술적 설명



1. 미션 소개

Python에서 API를 호출한다는 게 처음엔 막막하게 느껴집니다. 그런데 한 번 연결되는 순간, 가능성이 달라집니다.

LLM API가 추천을 하고, 지도 API가 위치를 찾고, 최종 리포트가 생성되는 흐름 — 이걸 손으로 조합합니다.

단일 API 호출이 아니라 여러 API를 엮어 인사이트를 만드는 과정, 그게 이번 미션입니다.

현대 서비스의 대부분은 API를 통해 데이터를 주고받습니다. AI 기능, 지도 데이터 등 다양한 서비스를 API로 손쉽게 연동할 수 있습니다.

이 미션에서는 LLM API와 지도 API를 조합하여 국내 여행 추천 프로그램을 개발합니다. 사용자가 여행 날짜를 입력하면, LLM이 해당 시기에 여행하기 좋은 지역을 추천하고, 지도 API로 맛집 정보를 검색한 뒤, 최종 여행 리포트를 생성합니다.

단순히 하나의 API를 호출하는 것을 넘어, 여러 API의 데이터를 조합하고 AI로 인사이트를 도출하는 프로그램을 만들어봅니다.

2. 최종 결과물

다음 3가지 결과물이 포함된 프로그램 1개를 완성한다.

1. CLI 기반 Python 프로그램

- 입력/요청: `-date "YYYY-MM-DD"` (필수)

- 출력/화면: 진행 로그 + 결과 저장 경로 안내

2. 실행 결과 데이터

- 출력/화면: results/ 폴더에 아래 파일이 생성되어야 한다.
- 원본 데이터 JSON 1개 이상 (1차 추천 결과 + 맛집 검색 결과를 포함)
- 최종 여행 리포트 Markdown 1개

3. README.md

- 프로그램 개요, 실행 방법, API 키 설정 방법, 결과물 확인 방법
- API 키가 유출되지 않도록 주의 사항 포함

3. 과제 목표

이 과제를 마친 후, 학습자는 아래를 스스로 설명할 수 있어야 한다.

- REST API의 요청/응답 구조와 HTTP 메서드(GET/POST)의 차이를 말로 설명할 수 있다.
- LLM 출력 결과를 구조화(JSON)하여 다음 단계(지도/장소 검색)의 입력으로 활용하는 흐름을 설명할 수 있다.
- 외부 API 호출에서 발생하는 대표 오류(인증/쿼터/네트워크/파싱)와 대응 원칙을 설명할 수 있다.
- API 키를 코드에 직접 작성하지 않고 .env /환경변수로 관리하는 이유를 설명할 수 있다.

4. 기능 요구 사항

다음 요구사항을 모두 만족해야 한다.

1. CLI 인터페이스

- argparse 를 활용하여 CLI로 실행 가능해야 한다.
- 필수 옵션: -date "YYYY-MM-DD"
- 입력값 검증: 날짜 형식이 올바르지 않으면 사용법을 출력하고 종료한다.

2. API 제공자 선택 규칙

- LLM API는 아래 중 택1로 구현한다.
 - OpenAI 계열 API
 - Google Gemini 계열 API

- 지도/장소 검색 API는 아래 중 택1로 구현한다. (국내 장소 검색 가능해야 함)
 - Kakao Local(키워드/카테고리 기반 장소 검색)
 - Naver Local Search(지역/장소 검색)
- 다른 제공자 활용 시, 아래 “공통 조건”을 만족해야 한다.
 - 장소 검색이 가능하고, 응답을 JSON으로 받을 수 있어야 한다.
 - 최소 응답 필드(또는 이에 준하는 값)를 확보할 수 있어야 한다:
place_name , address , lat , lng (또는 x/y), url (가능한 경우)

3. LLM API 연동 - 날씨/행사 정보

- 입력: 사용자가 입력한 date
- 출력: 반드시 JSON으로 파싱 가능한 텍스트를 생성하도록 프롬프트를 설계한다.
- 1차 JSON 최소 스키마(필수 키/타입)
 - recommended_city : string (예: "제주", "강릉")
 - weather : string (해당 시기 일반적 날씨 요약)
 - events : array of string (행사/축제 후보 1~3개)
 - reason : string (추천 근거 2~4문장)
- 실제 날씨/행사 데이터의 “정확도”를 평가하는 미션이 아니며, 중요한 것은 “구조화된 출력”과 “다음 단계 입력으로의 연결”이다.

4. 지도/장소 검색 API 연동 - 맛집 검색

- 입력: 1차 JSON의 recommended_city
- 출력: 해당 도시(또는 키워드) 기준으로 맛집 N곳(권장 5곳)을 검색한다.
- 맛집 아이템 최소 필드
 - name : string
 - address : string
 - category : string (가능한 경우)
 - url : string (가능한 경우)
 - x , y 또는 lat , lng : number (가능한 경우)
- 인증/요청 최소 가이드
 - Kakao Local 예시: 인증 헤더에 키를 실어 요청하는 방식(401/403 발생 시 키/권한/도메인 설정을 점검)

- Naver Local Search 예시: 클라이언트 ID/Secret을 헤더로 보내는 방식 (401/403 발생 시 키 값/헤더명 오타를 점검)
- 검색 결과가 0건이면 프로그램이 중단되지 않아야 하며, “데이터 없음” 상태로 다음 단계(리포트 생성)로 진행한다.

5. LLM API 연동 - 최종 리포트 생성

- 입력: 1차 추천 JSON + 맛집 목록(0건일 수 있음)
- 출력: 최종 여행 리포트를 Markdown 텍스트로 생성한다.
- 리포트에는 아래 항목이 포함되어야 한다.
 - 추천 지역 + 추천 이유 요약
 - 날씨 요약
 - 행사/축제 목록
 - 맛집 리스트 (0건이면 “데이터 없음”으로 표기)
 - 1일 일정 제 안(오전/오후/저녁 정도의 수준)

6. 에러 처리

- try-except 로 API 호출/파싱 오류를 처리한다.
- 최소 정책
 - API 키 미설정: 즉시 종료 + 설정 방법 안내 출력
 - 지도/장소 API 실패(네트워크/인증/쿼터 등): 맛집 섹션을 “데이터 없음” 처리하고 리포트 생성은 계속 진행
 - LLM JSON 파싱 실패: 재요청 1회 또는 “필수 키만 다시 JSON으로 출력” 하도록 프롬프트를 수정해 재시도 1회
- 실패가 발생하면, 결과 리포트에 errors 섹션으로 요약을 남길 수 있도록 내부적으로 오류 목록을 관리한다(빈 리스트여도 무방).

7. API 키 관리(보안)

- API 키를 코드에 직접 작성하지 않는다.
- 환경변수 또는 .env 파일(예: python-dotenv 사용 가능)에서 키를 읽어온다.
- 제출물(README/로그/결과 파일)에 키가 노출되지 않도록 주의한다.

8. 결과 저장

- results/ 폴더를 생성하고, 실행 날짜 기준으로 파일을 저장한다.
- 원본 데이터 JSON에는 최소한 아래가 포함되어야 한다.

- 1차 추천 JSON(파싱 결과)
 - 맛집 검색 결과(리스트, 0건 가능)
 - 오류 요약(errors : array)
- 최종 리포트는 .md 파일로 저장한다.

5. 보너스 과제 (선택)

1. 복수 지역 추천

- 1차 추천에서 recommended_city 를 1개가 아니라 2~3개로 확장한다(예: recommended_cities: [...]).
- 각 지역에 대해 맛집을 검색하고, 리포트에 지역별로 정리한다.
- 배움 포인트: “반복 처리(루프) + API 요청 관리 + 결과 구조 설계”를 경험한다.

2. 결과 캐싱(간단 버전)

- 같은 -date 로 재실행할 때, 이미 저장된 원본 JSON이 있으면 API 호출을 건너뛰고 리포트를 재생성한다(또는 그대로 사용).
- 배움 포인트: 외부 API 비용/속도 최적화의 기본을 경험한다.

개발환경

6. 개발 환경

- 언어 및 실행 환경
 - Python 3.10 이상을 사용한다.
 - 터미널에서 실행 가능해야 한다(웹 UI 구현은 요구하지 않음).

제약조건

7. 제약 사항

- 보안(필수)

- API 키를 코드/README/결과물에 직접 작성하지 않는다.
- .env 또는 환경변수를 사용한다.
- 왜 필요한가(최소 설명):
 - 협업/공유 시 실수로 키가 공개되는 것을 막는다.
 - 키를 교체하더라도 코드를 수정하지 않아도 된다(운영/배포에 유리).
 - 과금/쿼터가 걸린 서비스에서 사고를 예방한다.
- 환경변수 설정 예시(최소)
 - macOS/Linux(현재 터미널 세션에만 적용):
 - `export OPENAI_API_KEY="YOUR_KEY"`
 - Windows PowerShell(현재 세션에만 적용):
 - `$env:OPENAI_API_KEY="YOUR_KEY"`
 - 위 예시는 “설정 방식” 참고용이며, 실제 키 값은 절대 제출물에 포함하지 않는다.
- 운영 안정성(권장)
 - 키 미설정이면 즉시 종료하고 설정 방법을 안내한다.
 - 지도/장소 API 실패 시에도 리포트 생성은 진행한다(맞집=데이터 없음).
 - LLM JSON 파싱 실패 시 재시도는 최대 1회만 허용한다(무한 재시도 금지).

Test Case

8. 결과 예시

아래는 정답이 아니라 참고 예시다. 실제 출력 방식은 달라도 된다.

- 프로그램 실행

```
python travel_planner.py --date "20XX-03-15"
```

CSS

```
[1/3] 1차 추천 생성 중(LLM)...
```

```
- recommended_city: "제주"
```

```
[2/3] 맛집 검색 중(지도/장소 API)...
```

```
- 맛집 5곳 검색 완료
```

[3/3] 최종 리포트 생성 중(LLM)...

- 리포트 생성 완료

완료! results/20XX-03-15_travel_plan.md 를 확인하세요.

- 날씨/행사 정보

CSS

```
{
  "recommended_city": "제주",
  "weather": "3월 중순 평균 15°C 내외, 바람이 있으나 비교적 온화함",
  "events": ["유채꽃 관련 지역 행사", "봄 시즌 지역 축제(예: 일정 변동 가능)",
  "reason": "3월 중순은 제주가 봄꽃을 즐기기 좋은 시기입니다. 항공/숙박도 성수기 대비 부담이 적고, 야외 활동에 무리가 적습니다."
}
```

- 검색 실패 케이스

CSS

[2/3] 맛집 검색 중...

- 검색 결과 0건(키워드/카테고리를 바꿔 재시도하지 않고 다음 단계로 진행)

리포트 내 표기:

CSS

맛집 추천

- 데이터 없음 (장소 검색 결과 0건)

원본 JSON의 오류 요약:

CSS

```
{
  "errors": [
    {
      "step": "place_search",
      "type": "EMPTY_RESULT",
      "message": "0 results for query=..."
    }
  ]
}
```

```
}
```

- 지도/장소 API 인증 실패(401/403) 케이스

CSS

[2/3] 맛집 검색 중...

- 오류: 인증 실패(401). 키 설정을 확인하세요.
- 맛집 섹션은 '데이터 없음'으로 처리하고 계속 진행합니다.

원본 JSON의 오류 요약:

CSS

```
{  
  "errors": [  
    {"step": "place_search", "type": "AUTH_ERROR", "message": "HTTP  
401"}  
  ]  
}
```

- 최종 Markdown 리포트 형식

CSS

```
# 20XX-03-15 국내 여행 추천 리포트  
## 추천 지역  
## 추천 이유  
## 날씨 요약  
## 행사/축제  
## 맛집 추천  
## (선택) 1일 일정 제안  
## 오류 요약(errors)
```